

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I Hemalakhmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering

1. System Decomposition & Analysis

Analyze an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. System Decomposition & Analysis

Reverse engineering an AI-based system begins with breaking it down into its fundamental components to understand how it works internally. The goal is to identify the architecture, modules, data flow, and algorithms used.

A typical AI system consists of **four major layers**:

1. **Input Layer** – Collects raw data such as user interactions, sensor readings, or images.
2. **Preprocessing Layer** – Cleans, transforms, and normalizes data.
3. **Model Layer** – Contains machine learning algorithms, feature extraction logic, and decision-making modules.
4. **Output Layer** – Produces predictions, recommendations, alerts, or classifications.

During decomposition, each module is analyzed individually. For example, in a recommendation system, the **data ingestion module** gathers user logs, the **feature engineering module** extracts user-item features, the **ML engine** performs similarity computation or matrix factorization, and the **ranking module** generates final recommendations.

Data flow analysis reveals how information moves between modules. Raw data flows from storage to preprocessing, then into the ML model, and finally to the output interface. Underlying algorithms such as clustering, regression, neural networks, or collaborative filtering are identified by observing system behavior, input-output patterns, and performance characteristics.

This decomposition helps understand hidden dependencies, design decisions, and performance bottlenecks, enabling further reconstruction or redesign.

2. Architecture Reconstruction

Architecture reconstruction is performed when the system is a **black box**, meaning the internal code is not visible. The system is analyzed based on its inputs, outputs, and observable behavior. The goal is to rebuild a conceptual architecture that explains how the system operates.

The reconstruction process begins by identifying **functional modules**. For example, in a fraud detection system, the modules may include:

- **Input Module** (transaction streams)
- **Preprocessing Module** (cleaning, feature extraction)
- **Detection Engine** (rule-based + ML models)
- **Decision Module** (risk scoring, alert generation)
- **Output Module** (notifications, dashboards)

Next, the **interaction between modules** is inferred. The input module sends data to preprocessing, which transforms it and forwards it to the detection engine. The detection engine evaluates anomalies and passes results to the decision module, which triggers alerts. The output module communicates results to users or external systems.

Diagrams such as **block diagrams**, **flowcharts**, and **module interaction diagrams** are used to represent the reconstructed architecture. These diagrams show data movement, control flow, and module dependencies. Architecture reconstruction helps understand system behavior, identify hidden modules, and detect integration points, making it easier to redesign or optimize the system.

3. Algorithm Identification

Reverse engineering a machine learning application involves identifying the algorithms and decision logic used internally. This is done by analyzing input-output behavior, performance characteristics, and system responses to different data patterns.

The first step is identifying **data preprocessing algorithms**. For example, an image classification system may use resizing, normalization, noise removal, and augmentation techniques. A text-based system may use tokenization, stemming, and TF-IDF vectorization.

Next, the **feature extraction algorithms** are identified. In image systems, convolutional filters, pooling layers, and edge detectors are common. In text systems, word embeddings or n-gram models may be used.

The core ML algorithm is identified by observing system behavior. If the system outputs probability scores for multiple classes, it may use **softmax classification**. If it clusters data, it may use **k-means**. If it predicts continuous values, it may use **regression**. Neural network-based systems show non-linear decision boundaries and require large training data.

Finally, the **decision-making logic** is analyzed. This includes thresholding, confidence scoring, rule-based filtering, or ensemble voting. By examining how the system reacts to edge cases, noise, or ambiguous inputs, the underlying decision rules can be inferred.

Algorithm identification helps understand model complexity, accuracy, and limitations, enabling further optimization or replacement.

4. Data Flow & Process Mapping

Data flow mapping involves analyzing how data moves through a distributed AI system, how it is processed at each stage, and how components communicate. This is crucial for understanding system performance, scalability, and bottlenecks.

A distributed AI system typically includes:

1. **Data Sources** – IoT sensors, satellite feeds, social media, logs.
2. **Ingestion Layer** – Kafka streams, REST APIs, batch uploads.
3. **Processing Layer** – PySpark RDD/DataFrame transformations, feature extraction, model inference.
4. **Fusion Layer** – Combining multi-source data using joins, aggregations, and spatial-temporal analysis.
5. **Decision Layer** – Risk scoring, anomaly detection, classification.
6. **Output Layer** – Alerts, dashboards, reports.

Data flow mapping identifies how raw data enters the system, how it is cleaned, how features are extracted, and how models generate predictions. Communication between components is analyzed through message passing, shared storage, or distributed coordination.

For example, in a disaster detection system, sensor agents send readings to the ingestion layer, which streams data to the processing layer. The fusion engine combines sensor data with satellite images and social media signals. Decision agents evaluate risk and send alerts to communication agents. Mapping this flow helps identify latency issues, redundant processing, and opportunities for optimization.

5. Improvement & Redesign

Reverse engineering enables identification of weaknesses in an intelligent system, allowing redesign for better scalability, efficiency, and performance.

Common issues include slow batch processing, high latency, poor scalability, outdated algorithms, and inefficient communication between components. Improvement strategies focus on modernizing architecture, optimizing algorithms, and enhancing distributed processing.

Scalability Improvements

- Replace monolithic architecture with microservices
- Use distributed storage (HDFS, Cassandra)
- Introduce load balancing and parallel processing

Efficiency Improvements

- Optimize RDD/DataFrame transformations
- Use caching and in-memory computation
- Replace slow algorithms with faster alternatives (e.g., switch from k-means to mini-batch k-means)

Performance Improvements

- Introduce GPU acceleration for ML models
- Use asynchronous communication between agents
- Implement streaming instead of batch processing

Redesign Strategies

- Add intelligent agents for decentralized decision-making
- Introduce reinforcement learning for adaptive optimization
- Use containerization (Docker/Kubernetes) for deployment
- Improve fault tolerance and recovery mechanisms

These enhancements result in faster processing, better accuracy, improved scalability, and more reliable system performance.